

Understanding Trajectron++: A Per-Trajectory Performance Analysis

Simon Drauz and Zoe Matrullo
May 18, 2026

Showcase
Figure

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions

- **Task:** forecast future (x, y) positions of agents (pedestrians, cyclists, vehicles) from observed past movements

[figure placeholder]

- **Task:** forecast future (x, y) positions of agents (pedestrians, cyclists, vehicles) from observed past movements
- Challenging due to **complex agent interactions** and influence of **static context**

[figure placeholder]

- **Task:** forecast future (x, y) positions of agents (pedestrians, cyclists, vehicles) from observed past movements
- Challenging due to **complex agent interactions** and influence of **static context**
- Essential for **safe path planning** and **collision avoidance** in autonomous vehicles

[figure placeholder]

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)
- ⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
 - ▶ LSTM per agent

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)
- ⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
 - ▶ LSTM per agent
 - ▶ edges encode agent-agent interactions

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)
- ⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
 - ▶ LSTM per agent
 - ▶ edges encode agent-agent interactions
- *CVAE decoder*

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)
- ⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
 - ▶ LSTM per agent
 - ▶ edges encode agent-agent interactions
- *CVAE decoder*
 - ▶ Outputs a distribution over future trajectories

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)
- ⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
 - ▶ LSTM per agent
 - ▶ edges encode agent-agent interactions
- *CVAE decoder*
 - ▶ Outputs a distribution over future trajectories
- Handles *heterogeneous* agent types jointly (pedestrian, bicycle, car) and integrates HD map context

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)
- ⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

- 1 000 urban driving scenes, Boston & Singapore

[figure placeholder]

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles

[figure placeholder]

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles
- Rich sensor suite: camera, LiDAR, radar, GPS, HD map

[figure placeholder]

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles
- Rich sensor suite: camera, LiDAR, radar, GPS, HD map
- Standard split: 700 / 150 / 150 (train / val / test)

[figure placeholder]

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles
- Rich sensor suite: camera, LiDAR, radar, GPS, HD map
- Standard split: 700 / 150 / 150 (train / val / test)
- **Scope:** we predict *pedestrian* trajectories only; all agent types serve as scene context

[figure placeholder]

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles
- Rich sensor suite: camera, LiDAR, radar, GPS, HD map
- Standard split: 700 / 150 / 150 (train / val / test)
- **Scope:** we predict *pedestrian* trajectories only; all agent types serve as scene context

[figure placeholder]

Prediction metrics:

ml_ade	most-likely ADE ← used
ml_fde	most-likely FDE
min_ade_5	best-of-5 ADE
nll_mean	negative log-likelihood

- Some trajectories are inherently hard, others trivially easy—but *why*?

- Some trajectories are inherently hard, others trivially easy—but *why*?
- A single average ADE/FDE gives no insight into reasons for failure and success

- Some trajectories are inherently hard, others trivially easy—but *why*?
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

- Some trajectories are inherently hard, others trivially easy—but *why*?
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

⇒ **Three research questions:**

- Some trajectories are inherently hard, others trivially easy—but *why*?
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

- 1 Which trajectory and scene characteristics affect model performance?

⇒ **Three research questions:**

- Some trajectories are inherently hard, others trivially easy—but *why*?
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

⇒ **Three research questions:**

- ① Which trajectory and scene characteristics affect model performance?
- ② What are the *success and failure modes* that explain model performance?

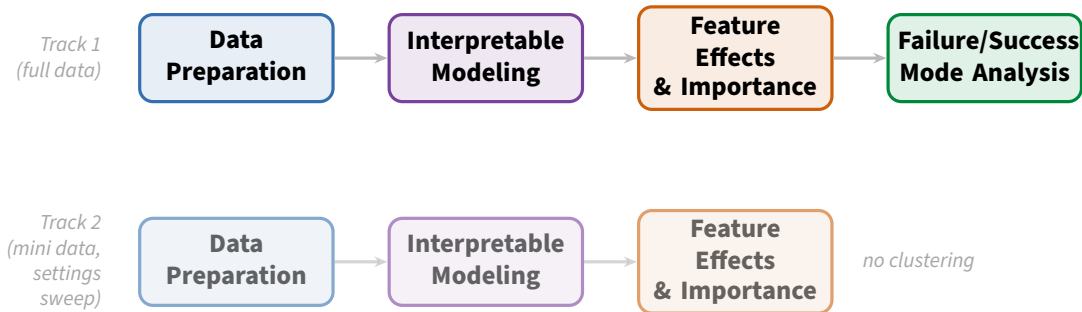
- Some trajectories are inherently hard, others trivially easy—but *why*?
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

⇒ **Three research questions:**

- ① Which trajectory and scene characteristics affect model performance?
- ② What are the *success and failure modes* that explain model performance?
- ③ How do model settings (history window, prediction horizon, attention radius) affect performance?

Outline

- 1 Motivation
- 2 Analysis Pipeline**
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions



Standard Trajectron++

- Evaluation outputs *batch-averaged*
ADE / FDE

Standard Trajectron++

- Evaluation outputs *batch-averaged*
ADE / FDE
- No way to link prediction error back to
an individual trajectory

Standard Trajectron++

- Evaluation outputs *batch-averaged*
ADE / FDE
- No way to link prediction error back to an individual trajectory
- Cannot study what makes a trajectory hard or easy

Standard Trajectron++

- Evaluation outputs *batch-averaged*
ADE / FDE
- No way to link prediction error back to an individual trajectory
- Cannot study what makes a trajectory hard or easy

Our Modification

Standard Trajectron++

- Evaluation outputs *batch-averaged* ADE / FDE
- No way to link prediction error back to an individual trajectory
- Cannot study what makes a trajectory hard or easy

Our Modification

- Recover *individual trajectory loss & characteristics* by modifying Trajectron++'s evaluation loop

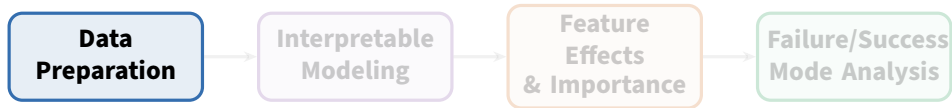
Standard Trajectron++

- Evaluation outputs *batch-averaged* ADE / FDE
- No way to link prediction error back to an individual trajectory
- Cannot study what makes a trajectory hard or easy

Our Modification

- Recover *individual trajectory loss & characteristics* by modifying Trajectron++'s evaluation loop
- `data_idx`: stable trajdata identifier, used to join evaluation output with computed trajectory & scene characteristics

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation**
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions



Agent motion features

Agent motion features

- Speed: mean, max, std
- Acceleration: mean, max
- Jerk: mean, max
- Heading change per second
- Min. neighbour distance
- Collision indicator

Agent motion features

- Speed: mean, max, std
- Acceleration: mean, max
- Jerk: mean, max
- Heading change per second
- Min. neighbour distance
- Collision indicator

Scene context features

Agent motion features

- Speed: mean, max, std
- Acceleration: mean, max
- Jerk: mean, max
- Heading change per second
- Min. neighbour distance
- Collision indicator

Scene context features

- Number of agents in scene
- Number of vehicles
- Scene bounding box area
- Spatial agent density

Agent motion features

- Speed: mean, max, std
- Acceleration: mean, max
- Jerk: mean, max
- Heading change per second
- Min. neighbour distance
- Collision indicator

Scene context features

- Number of agents in scene
- Number of vehicles
- Scene bounding box area
- Spatial agent density

⇒ **18 candidate features** joined with
Trajectron++ evaluation output via `data_idx`

- `ml_ade` is strongly *right-skewed* (skewness = 2.59)

[figure placeholder: before/after distribution]

- `ml_ade` is strongly *right-skewed* (skewness = 2.59)
- Small errors are very frequent; large errors are rare but influential

[figure placeholder: before/after distribution]

- `ml_ade` is strongly *right-skewed* (skewness = 2.59)
- Small errors are very frequent; large errors are rare but influential
- Log transformation ($\log_{10}$) applied to stabilise variance and improve model fit

[figure placeholder: before/after distribution]

- `ml_ade` is strongly *right-skewed* (skewness = 2.59)
- Small errors are very frequent; large errors are rare but influential
- Log transformation ($\log_{10}$) applied to stabilise variance and improve model fit
- Models are trained on log-scale target, evaluated on original scale

[figure placeholder: before/after distribution]

Approach 1: MI elbow + VIF

- Rank features by Mutual Information with target

Approach 1: MI elbow + VIF

- Rank features by Mutual Information with target
- Drop low-signal tail below the elbow: 18
→ 10

Approach 1: MI elbow + VIF

- Rank features by Mutual Information with target
- Drop low-signal tail below the elbow: 18 → 10
- Apply VIF filter (threshold = 5): 10 → **4 features**

Approach 1: MI elbow + VIF

- Rank features by Mutual Information with target
- Drop low-signal tail below the elbow: 18 $\rightarrow$ 10
- Apply VIF filter (threshold = 5): 10 $\rightarrow$ **4 features**

std_speed, mean_acceleration,
max_speed, heading_change_per_sec
Motion features only ($p \ll 0.01$).

Approach 1: MI elbow + VIF

- Rank features by Mutual Information with target
- Drop low-signal tail below the elbow: 18 $\rightarrow$ 10
- Apply VIF filter (threshold = 5): 10 $\rightarrow$ **4 features**

std_speed, mean_acceleration,
max_speed, heading_change_per_sec
Motion features only ($p \ll 0.01$).

Approach 2: VIF only

- Skip MI; apply VIF directly to all 18 candidates
- VIF filter (threshold = 5): 18 $\rightarrow$ **6 features**

Approach 1: MI elbow + VIF

- Rank features by Mutual Information with target
- Drop low-signal tail below the elbow: 18 $\rightarrow$ 10
- Apply VIF filter (threshold = 5): 10 $\rightarrow$ **4 features**

std_speed, mean_acceleration,
max_speed, heading_change_per_sec
Motion features only ($p \ll 0.01$).

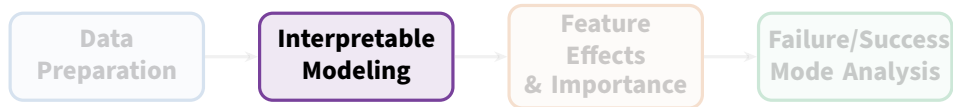
Approach 2: VIF only

- Skip MI; apply VIF directly to all 18 candidates
- VIF filter (threshold = 5): 18 $\rightarrow$ **6 features**

std_speed, mean_acceleration,
min_neighbor_distance,
heading_change_per_sec,
scene_num_VEHICLE, scene_density_VEHICLE
Retains scene-level features alongside motion features.

Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling**
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions



- We have per-trajectory `m1_ade` — but raw error values alone do not tell us *why* some trajectories are hard

- We have per-trajectory `ml_ade` — but raw error values alone do not tell us *why* some trajectories are hard
- A black-box model can predict ADE well but gives no insight into which features drive it or in which direction

- We have per-trajectory `ml_ade` — but raw error values alone do not tell us *why* some trajectories are hard
- A black-box model can predict ADE well but gives no insight into which features drive it or in which direction
- **Interpretable models** let us directly read off feature effects: how does speed, heading change, or scene density affect prediction error?

- We have per-trajectory `ml_ade` — but raw error values alone do not tell us *why* some trajectories are hard
- A black-box model can predict ADE well but gives no insight into which features drive it or in which direction
- **Interpretable models** let us directly read off feature effects: how does speed, heading change, or scene density affect prediction error?
- This turns ADE into actionable understanding of model behaviour

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

XGBoost

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms

XGBoost

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing

XGBoost

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing
- Directly readable smooth effects

XGBoost

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing
- Directly readable smooth effects

XGBoost

- Gradient-boosted decision trees

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing
- Directly readable smooth effects

XGBoost

- Gradient-boosted decision trees
- Flexibly captures non-linearities and feature interactions

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing
- Directly readable smooth effects

XGBoost

- Gradient-boosted decision trees
- Flexibly captures non-linearities and feature interactions
- Post-hoc interpretation via SHAP values

GAM — Additive Effects

- The model is a sum of smooth terms:

$$\hat{y} = \sum_j \hat{s}_j(x_j)$$

XGBoost — SHAP Values

GAM — Additive Effects

- The model is a sum of smooth terms:

$$\hat{y} = \sum_j \hat{s}_j(x_j)$$

- Each $\hat{s}_j$ directly shows how feature j affects `ml_ade` — no post-hoc step needed

XGBoost — SHAP Values

GAM — Additive Effects

- The model is a sum of smooth terms:

$$\hat{y} = \sum_j \hat{s}_j(x_j)$$

- Each $\hat{s}_j$ directly shows how feature j affects `ml_ade` — no post-hoc step needed
- Confidence bands from the spline fit

XGBoost — SHAP Values

GAM — Additive Effects

- The model is a sum of smooth terms:
$$\hat{y} = \sum_j \hat{s}_j(x_j)$$
- Each $\hat{s}_j$ directly shows how feature j affects `ml_ade` — no post-hoc step needed
- Confidence bands from the spline fit

XGBoost — SHAP Values

- Each prediction is decomposed into per-feature contributions post-hoc

GAM — Additive Effects

- The model is a sum of smooth terms:
$$\hat{y} = \sum_j \hat{s}_j(x_j)$$
- Each $\hat{s}_j$ directly shows how feature j affects `ml_ade` — no post-hoc step needed
- Confidence bands from the spline fit

XGBoost — SHAP Values

- Each prediction is decomposed into per-feature contributions post-hoc
- *Local*: why was this trajectory's ADE high?

GAM — Additive Effects

- The model is a sum of smooth terms:
 $\hat{y} = \sum_j \hat{s}_j(x_j)$
- Each $\hat{s}_j$ directly shows how feature j affects `ml_ade` — no post-hoc step needed
- Confidence bands from the spline fit

XGBoost — SHAP Values

- Each prediction is decomposed into per-feature contributions post-hoc
- *Local*: why was this trajectory's ADE high?
- *Global*: mean $|\text{SHAP}|$ gives feature importance and direction

- **Problem:** hyperparameter tuning risks overfitting to a particular data split
- **Solution:** nested cross-validation
 - ▶ Outer folds: evaluate generalization of the full tuning procedure
 - ▶ Inner folds: tune hyperparameters on each outer-fold training set via cross-validation
- OOF evaluation: inspect out-of-fold predictions for instability, outlier folds, bias
- **Conclusion:** tuning generalises stably → proceed to full-data tuning + refit

Final models are refit on *all* data with optimal hyperparameters — no held-out test set, because the goal is understanding, not unbiased prediction.

Three candidates:

- **LinearGAM** — fitted on raw `m1_ade`

Three candidates:

- **LinearGAM** — fitted on raw `m1_ade`
- **LinearGAM (log)** — fitted on `log(m1_ade)`, evaluated on original scale

Variant	RMSE	R^2	MAE
LinearGAM	0.440	0.515	0.268
LinearGAM (log)	0.444	0.506	0.255
GammaGAM	0.475	0.434	0.277

Outer-fold means, 3-fold nested CV.

LinearGAM selected.

Selection criterion: lowest mean outer-fold RMSE (original scale).

Three candidates:

- **LinearGAM** — fitted on raw `m1_ade`
- **LinearGAM (log)** — fitted on `log(m1_ade)`, evaluated on original scale
- **GammaGAM** — Gamma distribution with log link, suited for positive right-skewed targets

Selection criterion: lowest mean outer-fold RMSE (original scale).

Variant	RMSE	R ²	MAE
LinearGAM	0.440	0.515	0.268
LinearGAM (log)	0.444	0.506	0.255
GammaGAM	0.475	0.434	0.277

Outer-fold means, 3-fold nested CV.

LinearGAM selected.

GAM (LinearGAM)

- Fitted on raw `ml_ade`
- Additive smooth terms, one per feature

Metric	OOF value
RMSE	0.440
R^2	0.515
MAE	0.268

XGBoost

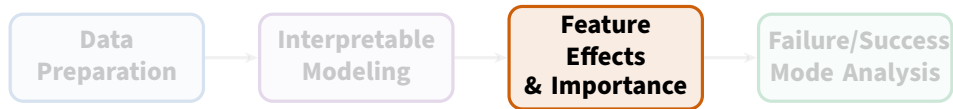
- Gradient-boosted trees
- Post-hoc interpretation via SHAP

Metric	OOF value
RMSE	0.387
R^2	0.624
MAE	0.213

XGBoost outperforms GAM on all metrics. Both models are used as a cross-check: the feature-effect narratives should agree across approaches.

Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance**
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions



XGBoost — mean |SHAP|

Feature	Mean SHAP
std_speed	0.129
heading_change_per_sec	0.070
min_neighbor_distance	0.027
mean_acceleration	0.027
scene_density_VEHICLE	0.020
scene_num_VEHICLE	0.016

GAM — smooth-term significance

Feature	$p < 0.05?$
heading_change_per_sec	✓
mean_acceleration	✓
min_neighbor_distance	✓
std_speed	✓
scene_num_VEHICLE	✓
scene_density_VEHICLE	×

Both models agree: speed variation (std_speed) and turning (heading_change_per_sec) are the dominant drivers of prediction error. Scene density is consistently least important.

Motion features

- `std_speed`: strong positive effect — erratic speed changes make trajectories harder to forecast

Motion features

- `std_speed`: strong positive effect — erratic speed changes make trajectories harder to forecast
- `heading_change_per_sec`: positive effect — more turning → higher ADE

Motion features

- `std_speed`: strong positive effect — erratic speed changes make trajectories harder to forecast
- `heading_change_per_sec`: positive effect — more turning → higher ADE
- `mean_acceleration`: positive effect — higher acceleration raises predicted error

Scene & proximity features

- `min_neighbor_distance`: mild positive effect — isolated agents are marginally harder to predict
- `scene_num_VEHICLE`: small positive effect; crowded vehicle context increases error slightly
- `scene_density_VEHICLE`: not significant in GAM; weakest SHAP contribution in XGBoost

Motion features

- `std_speed`: strong positive effect — erratic speed changes make trajectories harder to forecast
- `heading_change_per_sec`: positive effect — more turning → higher ADE
- `mean_acceleration`: positive effect — higher acceleration raises predicted error

Scene & proximity features

- `min_neighbor_distance`: mild positive effect — isolated agents are marginally harder to predict
- `scene_num_VEHICLE`: small positive effect; crowded vehicle context increases error slightly
- `scene_density_VEHICLE`: not significant in GAM; weakest SHAP contribution in XGBoost

⇒ Trajectron++ struggles most with **fast, erratic, turning pedestrians**; scene context is a secondary factor.

GAM — smooth effects

- Marginal effect of each feature on predicted `ml_ade`, holding others fixed

GAM — smooth effects

- Marginal effect of each feature on predicted `ml_ade`, holding others fixed
- `std_speed`: monotonically increasing — more speed variation → higher error

XGBoost — SHAP dependence

- Global picture consistent with GAM: `std_speed` contributes most strongly
- `min_neighbor_distance` shows a non-monotonic pattern — very close *and* very distant neighbors raise error

GAM — smooth effects

- Marginal effect of each feature on predicted `m1_ade`, holding others fixed
- `std_speed`: monotonically increasing — more speed variation → higher error
- `heading_change_per_sec`: positive, non-linear — effect levels off at high turning rates

XGBoost — SHAP dependence

- Global picture consistent with GAM: `std_speed` contributes most strongly
- `min_neighbor_distance` shows a non-monotonic pattern — very close *and* very distant neighbors raise error
- Both models: scene features show flat or near-zero marginal effects compared to motion features

GAM — smooth effects

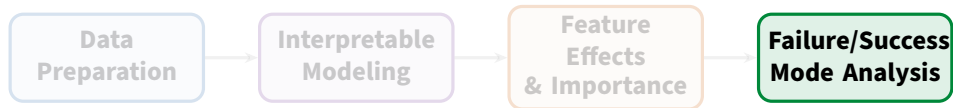
- Marginal effect of each feature on predicted `m1_ade`, holding others fixed
- `std_speed`: monotonically increasing — more speed variation → higher error
- `heading_change_per_sec`: positive, non-linear — effect levels off at high turning rates
- Confidence bands narrow in high-density regions

XGBoost — SHAP dependence

- Global picture consistent with GAM: `std_speed` contributes most strongly
- `min_neighbor_distance` shows a non-monotonic pattern — very close *and* very distant neighbors raise error
- Both models: scene features show flat or near-zero marginal effects compared to motion features

Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes**
- 7 Model Settings Analysis
- 8 Conclusions



- ① Divide trajectories into difficulty groups based on `m1_ade` (easy / medium / hard)
- ② Run clustering on trajectory + scene characteristics per group
 - ▶ Option A: original feature space
 - ▶ Option B: UMAP-reduced space (dimensions chosen via trustworthiness score + elbow rule)
- ③ Assess clustering quality with DBCV score
- ④ Export identified clusters (performance regimes)

Performance group sizes (VIF-only run)

Group	Boundary	N	%
Easy	< 0.39	15 348	59%
Medium	0.39–1.20	8 094	31%
Hard	> 1.20	2 680	10%

Performance group sizes (VIF-only run)

Group	Boundary	N	%
Easy	< 0.39	15 348	59%
Medium	0.39–1.20	8 094	31%
Hard	> 1.20	2 680	10%

- Clustering algorithm: **OPTICS** on raw feature-effect space (no UMAP needed for 6-dimensional space)
- Quality assessed via **DBCv** score

Performance group sizes (VIF-only run)

Group	Boundary	N	%
Easy	< 0.39	15 348	59%
Medium	0.39–1.20	8 094	31%
Hard	> 1.20	2 680	10%

- Clustering algorithm: **OPTICS** on raw feature-effect space (no UMAP needed for 6-dimensional space)
- Quality assessed via **DBC**V score

Cluster quality by group

Group	DBC	V	Noise %
Easy	0.411		30%
Medium	0.397		40%
Hard	0.091		75%

Easy group has

the clearest substructure.

Hard is mostly a continuous high-error regime.

Easy regime

- Low `std_speed` (-2.0σ global) and low heading change $\rightarrow$ slow, predictable trajectories

Easy regime

- Low `std_speed` (-2.0σ global) and low heading change $\rightarrow$ slow, predictable trajectories
- Two sub-clusters: *slow/stationary* (median ADE 0.02) vs. *mild motion* (median ADE 0.22)

Hard regime

- 99.5% of hard trajectories have *positive* total feature effect
- Dominated by high `std_speed` ($+2.0\sigma$ global), turning, and acceleration

Easy regime

- Low std_speed (-2.0σ global) and low heading change $\rightarrow$ slow, predictable trajectories
- Two sub-clusters: *slow/stationary* (median ADE 0.02) vs. *mild motion* (median ADE 0.22)
- Both drive negative total feature effects

Hard regime

- 99.5% of hard trajectories have *positive* total feature effect
- Dominated by high std_speed ($+2.0\sigma$ global), turning, and acceleration
- No sharp sub-clusters — hard is a **continuous high-variation regime**; a small extreme outlier group has $\text{std_speed} \approx +4.8\sigma$

Takeaway: Trajectron++ struggles with erratic, fast-turning pedestrians. Slow or near-stationary pedestrians are easy regardless of scene complexity.

Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis**
- 8 Conclusions

- Dataset: nuScenes mini (full dataset too expensive for repeated runs)
- Cartesian sweep over:
 - ▶ `history_sec`
 - ▶ `prediction_sec`
 - ▶ `attention_radius_m`
- Pooled model: model setting variables + trajectory/scene features as control variables
- Feature normalization: setting-agnostic versions to avoid mechanical confounding (e.g. `mean_path_length_per_second` instead of `path_length`)

XGBoost — mean |SHAP| (pooled model)

Feature	Mean SHAP	All 8
prediction_sec	0.178	
attention_radius_m	0.098	
std_speed	0.093	
max_speed	0.070	
history_sec	0.022	

features significant in GAM ($p \ll 0.01$).

Key findings

- prediction_sec: largest effect — longer horizon means harder forecasting (ADE grows with time)

XGBoost — mean |SHAP| (pooled model)

Feature	Mean SHAP	All 8
prediction_sec	0.178	
attention_radius_m	0.098	
std_speed	0.093	
max_speed	0.070	
history_sec	0.022	

features significant in GAM ($p \ll 0.01$).

Key findings

- prediction_sec: largest effect — longer horizon means harder forecasting (ADE grows with time)
- attention_radius_m: second — larger radius provides more context but also adds noise; non-linear optimal range

XGBoost — mean |SHAP| (pooled model)

Feature	Mean SHAP	All 8
prediction_sec	0.178	
attention_radius_m	0.098	
std_speed	0.093	
max_speed	0.070	
history_sec	0.022	

features significant in GAM ($p \ll 0.01$).

Key findings

- prediction_sec: largest effect — longer horizon means harder forecasting (ADE grows with time)
- attention_radius_m: second — larger radius provides more context but also adds noise; non-linear optimal range
- history_sec: smallest setting effect — model is robust to history window length in this range

XGBoost — mean |SHAP| (pooled model)

Feature	Mean SHAP	All 8
prediction_sec	0.178	
attention_radius_m	0.098	
std_speed	0.093	
max_speed	0.070	
history_sec	0.022	

features significant in GAM ($p \ll 0.01$).

Key findings

- prediction_sec: largest effect — longer horizon means harder forecasting (ADE grows with time)
- attention_radius_m: second — larger radius provides more context but also adds noise; non-linear optimal range
- history_sec: smallest setting effect — model is robust to history window length in this range
- Trajectory features (std_speed, max_speed) retain strong explanatory power even after controlling for settings

Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions**

① RQ1 — Which characteristics drive prediction error?

- ▶ `std_speed` (speed variation) and `heading_change_per_sec` (turning) are the dominant drivers in both GAM and XGBoost
- ▶ Scene-level features (`scene_density_VEHICLE`) are consistently least important; Trajectron++ handles scene context better than individual motion dynamics

① RQ1 — Which characteristics drive prediction error?

- ▶ `std_speed` (speed variation) and `heading_change_per_sec` (turning) are the dominant drivers in both GAM and XGBoost
- ▶ Scene-level features (`scene_density_VEHICLE`) are consistently least important; Trajectron++ handles scene context better than individual motion dynamics

② RQ2 — What are the success and failure modes?

- ▶ Easy: slow or near-stationary pedestrians with low speed variation — Trajectron++ excels here
- ▶ Hard: continuous high-variation regime driven by erratic, fast-turning pedestrians; no sharp sub-clusters

① RQ1 — Which characteristics drive prediction error?

- ▶ `std_speed` (speed variation) and `heading_change_per_sec` (turning) are the dominant drivers in both GAM and XGBoost
- ▶ Scene-level features (`scene_density_VEHICLE`) are consistently least important; Trajectron++ handles scene context better than individual motion dynamics

② RQ2 — What are the success and failure modes?

- ▶ Easy: slow or near-stationary pedestrians with low speed variation — Trajectron++ excels here
- ▶ Hard: continuous high-variation regime driven by erratic, fast-turning pedestrians; no sharp sub-clusters

③ RQ3 — How do model settings affect performance?

- ▶ Prediction horizon (`prediction_sec`) has the largest effect — longer horizon raises error strongly
- ▶ Attention radius (`attention_radius_m`) is second; history window (`history_sec`) has minimal impact

Limitations

- Analysis restricted to *pedestrian* trajectories on nuScenes — generalisability to other agent types or datasets is not guaranteed

Limitations

- Analysis restricted to *pedestrian* trajectories on nuScenes — generalisability to other agent types or datasets is not guaranteed
- Meta-models explain Trajectron++ predictions, not ground-truth difficulty — effects are model-specific

Limitations

- Analysis restricted to *pedestrian* trajectories on nuScenes — generalisability to other agent types or datasets is not guaranteed
- Meta-models explain Trajectron++ predictions, not ground-truth difficulty — effects are model-specific
- Hard-regime clustering is weak (75% noise); discrete failure-mode identification remains an open challenge

Future Work

Limitations

- Analysis restricted to *pedestrian* trajectories on nuScenes — generalisability to other agent types or datasets is not guaranteed
- Meta-models explain Trajectron++ predictions, not ground-truth difficulty — effects are model-specific
- Hard-regime clustering is weak (75% noise); discrete failure-mode identification remains an open challenge
- Settings sweep uses nuScenes *mini* — results may not transfer to full-scale training

Future Work

- Extend pipeline to vehicle and cyclist trajectories
- Apply to other trajectory prediction models and datasets via the trajdata interface
- Explore higher-capacity meta-models to better isolate failure modes in the hard regime